

Requerimientos Implementados

Parte A – Comunicación entre procesos

Se desarrolló una estructura básica para la comunicación entre componentes del sistema distribuido mediante la clase clienteNodo.

Código implementado:

```
package org.uteq.practica_evaluacion.model;  
  
import org.uteq.practica_evaluacion.NodeConfig;  
  
public class clienteNodo {  
    private final NodeConfig config;  
  
    public clienteNodo(NodeConfig config) {  
        this.config = config;  
    }  
  
    public void message(message.tipo tipo) {  
        System.out.println("Mensaje recibido: " + tipo);  
    }  
}
```

Explicación:

La clase permite recibir mensajes y mostrarlos por consola. Esto constituye una base para la comunicación entre nodos y clientes dentro del sistema distribuido.

Parte B – Relojes lógicos de Lamport

Se creó la clase destinada a implementar el algoritmo de Lamport.

Código implementado

```
public class implementLamport {  
}
```

Explicación:

Durante la práctica se creó la estructura de la clase encargada de gestionar los relojes lógicos de Lamport. Sin embargo, la lógica del algoritmo no fue implementada completamente.

Parte C – Tolerancia a fallos

Para la detección de fallos se definieron parámetros relacionados con el envío y control de heartbeats.

Código implementado

```
public class NodeConfig {  
    private int id;  
    private String peers;  
    private long heartbeatIntervalMs = 2000;  
    private long heartbeatTimeoutMs = 6000;  
}
```

Explicación:

Se configuraron intervalos de tiempo para el envío de heartbeats y para la detección de posibles fallos.

Parte D – Coordinación

Se definieron los tipos de mensajes necesarios para soportar el algoritmo de elección de líder.

Código implementado

```
public enum tipo {  
    OPERATION,  
    REPLICATE,  
    HEARTBEAT,  
    HEARTBEAT_ACK,  
    ELECTION,  
    OK,  
    COORDINATOR,  
    REJECT  
}
```

Explicación:

Los mensajes ELECTION, OK y COORDINATOR fueron definidos para representar las operaciones necesarias en un proceso de elección de coordinador.

Parte E – Seguridad

Actualmente no se implementó un mecanismo de autenticación mediante token.

Explicación:

La estructura del proyecto fue preparada para futuras ampliaciones; sin embargo, la validación de tokens y el control de acceso no fueron desarrollados durante esta práctica.

4. Resultados Obtenidos

Durante la práctica se logró construir la estructura inicial del sistema distribuido, definiendo componentes para la comunicación, configuración de nodos y tipos de mensajes necesarios para mecanismos de sincronización, coordinación y tolerancia a fallos.

5. Conclusiones

El desarrollo realizado permitió adquirir experiencia en la organización de proyectos distribuidos utilizando Java y Spring Boot, identificando las dificultades técnicas que surgen durante la implementación de mecanismos como sincronización, coordinación y tolerancia a fallos.